tmx-pane-shim.sh — split-topology pane launcher

Revision: 1 **Last modified:** 2026-07-23T00:00:00Z

Overview

`scripts/tmx-pane-shim.sh` is the pane launcher of the §11.4.225 interactive-server scope split (`TMX_SERVER_SPLIT=1`, opt-in, default OFF). In the split topology the tmux SERVER runs alone in `tmx-<name>.scope` with a small host-adaptive CPU quota, while every pane's shell — and therefore the entire workload fleet a pane spawns — runs inside its own transient systemd scope under the session's dedicated workload slice `tmxw-<name>.slice` (dashes in the session name are `\x2d`-escaped so every session slice is a flat leaf — no accidental slice nesting between sessions whose names prefix each other). The slice carries the remainder of the session CPU budget plus the CFS burst bank, so fleet bursts can never starve the server's keystroke/echo/timer handling (HEL-006/HEL-003 forensics, 2026-07-22/23).

Prerequisites

- Linux with systemd ≥ 230 and a functional `systemd-run --user --scope` (the same preconditions as the shared-scope topology).
- The pane environment must reach the user manager (`$XDG_RUNTIME_DIR/bus` — inherited from the tmux server's environment; true for every wrapper-spawned session).
- The wrapper (`scripts/tmx`, generated from `scripts/tmx.template`) passes the slice unit as the single argument; operators never invoke the shim directly.

Usage

```bash
# outer stage — what tmux runs as the window command / default-command:
bash tmx-pane-shim.sh 'tmxw-mysession.slice'
```

```bash
# inner stage — internal only, executed via systemd-run inside the
        pane scope:
bash tmx-pane-shim.sh --inner 'tmxw-mysession.slice' /tmp/.tmx-
        pane-shim.XXXXXX
```

Internal behaviour

1. **Outer stage** creates a placement marker file, then runs `systemd-run --user --scope --collect --quiet --slice=<slice> -- bash <self> --inner <slice> <marker>`. When systemd-run returns, the marker decides: marker written ⇒ the shell genuinely ran inside the slice and has exited — propagate its exit code (the pane closes normally). Marker empty ⇒ scope creation failed — print a loud FATAL, hold the message 5 s, exit 7.
2. **Inner stage** re-reads `/proc/self/cgroup` (the KERNEL's record, §11.4.200 verify-after-write against the authoritative source) and aborts unless the path names the slice; only then writes the marker and `exec`s the user's login shell (`$SHELL -l`, fallback `/bin/bash`).

Fail-closed contract (§11.4.200/ §11.4.201)

A pane that cannot be placed inside the workload slice must NEVER silently run outside it — that would put the fleet into the server's small scope (or an unbounded cgroup), strictly worse than the shared topology. The shim refuses; additionally the wrapper's post-create verify polls the slice cgroup for a live pane pid and aborts the whole session (teardown + exit 1) if the first pane never lands.

Edge cases

- `systemd-run` missing from the pane environment → loud FATAL, exit 7.
- The slice was torn down between pane creation and shim start → scope creation fails → marker empty → loud FATAL (never a silent fallback).
- `mktemp` failure → marker-based detection degrades gracefully (the inner cgroup verification still guards placement).
- Shared topology (`TMX_SERVER_SPLIT` unset/0): the shim is not invoked at all.

Related scripts

- `scripts/tmx.template` — spawner: split activation, slice `set-property`, post-create verify, pair teardown.
- `scripts/tmx-recycler.sh` — idle recycler; stops the workload slice (`TMX_RC_SLICE`) alongside the scope.
- `scripts/tests/87_server_scope_split.sh` — RED/GREEN polarity test + live throttle-isolation A/B.
- `scripts/tests/09_crash_isolation_scope.sh` (T7) / `15_per_session_cgroup_distinct.sh` (T7–T9) — split-topology crash-isolation and pair-distinctness regression sections.

Last verified

2026-07-23 — live on systemd 255 (64-core Linux host): placement, fail-closed abort, throttle isolation (slice 30 throttled periods under an 8-spinner fleet vs server scope 0), pair teardown.